Aula Extra E2 — Redução de Dimensionalidade (PCA e t-SNE)

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME cap. 10; ISLP §12.2

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- listar as razões para reduzir a dimensionalidade;
- definir os **componentes principais** como direções de variância máxima e resolvê-los via autovetores;
- usar a *proporção de variância explicada* para escolher quantos componentes manter;
- interpretar o PCA geometricamente e usá-lo para **compressão**;
- reconhecer as extensões não lineares — **Kernel PCA** e **t-SNE** — e os cuidados com este último.

Esta aula ataca de frente a maldição da dimensionalidade, mas pelo lado da *redundância*: se as covariáveis são correlacionadas, os dados ocupam apenas uma fatia fina do espaço $\mathbb{R}^p$, e talvez baste descrevê-la. A pergunta:

como resumir p covariáveis em bem menos, sem perder o essencial?

1 Por que reduzir a dimensionalidade?

- **Visualização**: projetar dados em 2D/3D para enxergar estrutura.
- **Compressão**: armazenar/transmitir com menos espaço (imagens).
- **Pré-processamento**: combater a *maldição da dimensionalidade* e o ruído antes de um método supervisionado.
- **Redundância**: muitas covariáveis correlacionadas vivem perto de uma subvariedade de dimensão menor — queremos recuperá-la.

Ideia central

O último item é o mais importante conceitualmente, e liga esta aula ao que vimos em regressão. O KNN *se adapta sozinho* à dimensão intrínseca, mas métodos que somam todas as coordenadas sofrem com p grande. O PCA faz esse trabalho *explicitamente*: ele encontra a subvariedade — desde que ela seja aproximadamente um subespaço linear.

2 Componentes principais (PCA)

Suponha as covariáveis *centradas* (média zero). O **primeiro componente principal** é a combinação linear

$$Z_1 = \phi_{11} X_1 + \cdots + \phi_{p1} X_p = \boldsymbol{\phi}_1^\top \boldsymbol{X}$$

de **variância máxima**, sob a restrição $\|\boldsymbol{\phi}_1\|^2 = \sum_k \phi_{k1}^2 = 1$ (sem ela, bastaria inflar os coeficientes). O i-ésimo componente Z_i tem variância máxima entre as combinações lineares unitárias *não correlacionadas* com $Z_1, \ldots, Z_{i-1}$.

Ideia central

Procuramos novos eixos (ortogonais) ao longo dos quais os dados *mais variam*. O 1º eixo capta o máximo de variabilidade; o 2º, o máximo do que resta sendo ortogonal ao 1º; e assim por diante. Concentrar a variância em poucos eixos é o que permite descartar os demais com pouca perda.

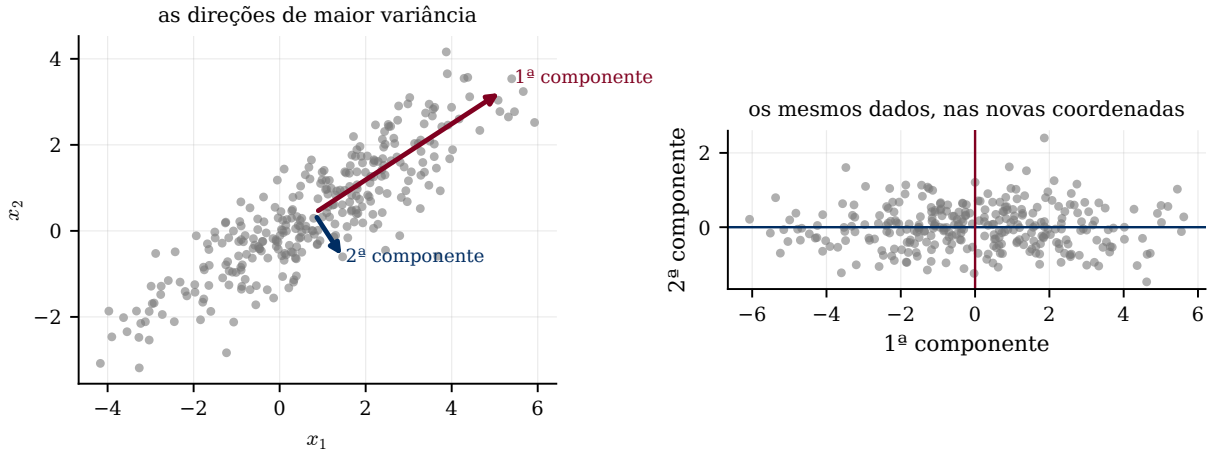


Figura 1: PCA em duas dimensões. À esquerda, a nuvem original e as duas direções principais, com comprimento proporcional ao desvio-padrão ao longo delas: a primeira acompanha o alongamento da nuvem e responde por 94,4% da variância. À direita, os mesmos pontos escritos nas novas coordenadas — é literalmente uma *rotação*. E é aí que a redução fica óbvia: descartar a segunda coordenada custa 5,6% da variância e transforma dois números em um.

2.1 Solução: autovetores da covariância

Seja $\boldsymbol{C} = \boldsymbol{X}^\top \boldsymbol{X}$ (proporcional à matriz de covariância, pois os dados têm média zero). O problema é

$$\max_{\boldsymbol{\phi}} \ \mathrm{Var}(\boldsymbol{\phi}^\top \boldsymbol{X}) = \max_{\boldsymbol{\phi}} \ \boldsymbol{\phi}^\top \boldsymbol{C} \boldsymbol{\phi} \quad \text{s.a.} \quad \|\boldsymbol{\phi}\|^2 = 1.$$

Proposição 2.1 (O 1º componente é o autovetor principal)**.** *O maximizador de $\boldsymbol{\phi}^\top \boldsymbol{C} \boldsymbol{\phi}$ sob $\|\boldsymbol{\phi}\| = 1$ é o autovetor de $\boldsymbol{C}$ associado ao* maior *autovalor, e o valor máximo da variância é esse autovalor.*

Demonstração. Lagrangiano: $\mathcal{L}(\boldsymbol{\phi}, \nu) = \boldsymbol{\phi}^\top \boldsymbol{C} \boldsymbol{\phi} - \nu(\boldsymbol{\phi}^\top \boldsymbol{\phi} - 1)$. Anulando o gradiente em $\boldsymbol{\phi}$: $2\boldsymbol{C}\boldsymbol{\phi} - 2\nu\boldsymbol{\phi} = 0$, isto é, $\boldsymbol{C}\boldsymbol{\phi} = \nu\boldsymbol{\phi}$ — $\boldsymbol{\phi}$ é *autovetor* de $\boldsymbol{C}$ com autovalor ν. Para um autovetor unitário, a variância é $\boldsymbol{\phi}^\top \boldsymbol{C} \boldsymbol{\phi} = \nu$. Logo o máximo é atingido no autovetor de *maior* autovalor. Os componentes seguintes saem do mesmo argumento restrito ao complemento ortogonal, gerando os demais autovetores em ordem decrescente de autovalor. □

Ideia central

Vale apreciar quanto essa proposição entrega. O problema original — "ache a melhor direção, depois a melhor entre as ortogonais a ela, e assim por diante" — parece uma sequência de p otimizações encadeadas. A proposição diz que **uma única** decomposição espectral de $\boldsymbol{C}$ resolve todas de uma vez, e ainda ordena o resultado. É por isso que o PCA é barato mesmo com p grande.

Empilhando os autovetores (ordenados por autovalor decrescente) nas colunas de $\boldsymbol{U}$ (a **matriz de cargas**), os *scores* são

$$\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{U} \quad (n \times p),$$

e a i-ésima nova coordenada da observação $\boldsymbol{x}$ é $z_i = \boldsymbol{u}_i^\top \boldsymbol{x}$. Como os autovetores são ortogonais, os componentes são *não correlacionados* — sem informação redundante.

2.2 Quantos componentes? Variância explicada

Cada autovalor λ_i é a variância capturada pelo i-ésimo componente. A **proporção de variância explicada** pelos k primeiros é

$$\mathrm{PVE}(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{p} \lambda_i}. \tag{1}$$

Escolhe-se k olhando o *scree plot* (λ_i vs. i, procurando o "cotovelo") ou exigindo PVE acima de um limiar (ex.: 90%).

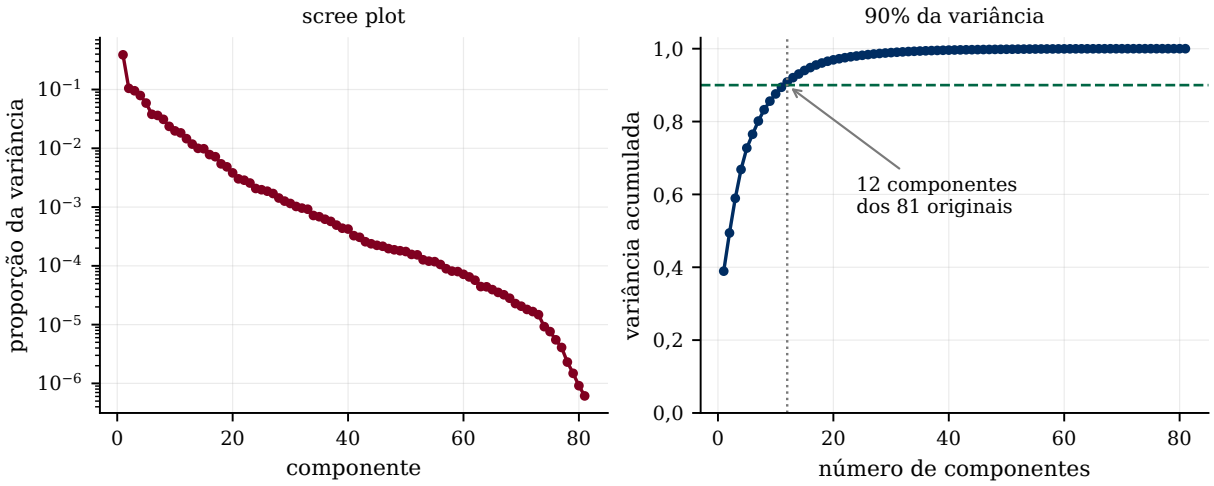


Figura 2: PCA nas 81 covariáveis padronizadas do `superconductivity.csv`, a mesma base das aulas práticas. O primeiro componente sozinho explica 38,9% da variância, e **12 componentes** chegam a 90%. Em outras palavras: as 81 colunas carregam cerca de uma dúzia de dimensões de informação, e as demais 69 são, em boa medida, repetição. É a *redundância*, medida.

3 Interpretação geométrica e compressão

3.1 PCA como escalonamento multidimensional

O PCA também resolve: encontrar a projeção linear $\boldsymbol{Z}$ em $k < p$ dimensões que melhor *preserva as distâncias* entre pares,

$$\min_{\boldsymbol{Z}} \sum_{i,j} \left(\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 - \|\boldsymbol{z}_i - \boldsymbol{z}_j\|^2 \right).$$

A solução são os k primeiros componentes principais. Ou seja, PCA é a melhor aproximação linear de baixa dimensão no sentido de distâncias — daí ser tão útil para visualização. Note que são *dois* problemas aparentemente diferentes (maximizar variância, preservar distâncias) com a *mesma* solução; guarde isso, porque o t-SNE vai atacar o segundo problema por outro caminho.

3.2 Compressão (SVD truncada)

Da decomposição $\boldsymbol{X}^\top \boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{U}^\top$, reconstrói-se $\boldsymbol{X} \approx \boldsymbol{Z}_k \boldsymbol{U}_k^\top$, usando só as k primeiras colunas. Se os autovalores decaem rápido, a reconstrução é quase perfeita guardando apenas $(z_{i,1}, \ldots, z_{i,k})$ e as k cargas $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_k$ — uma **compressão**.

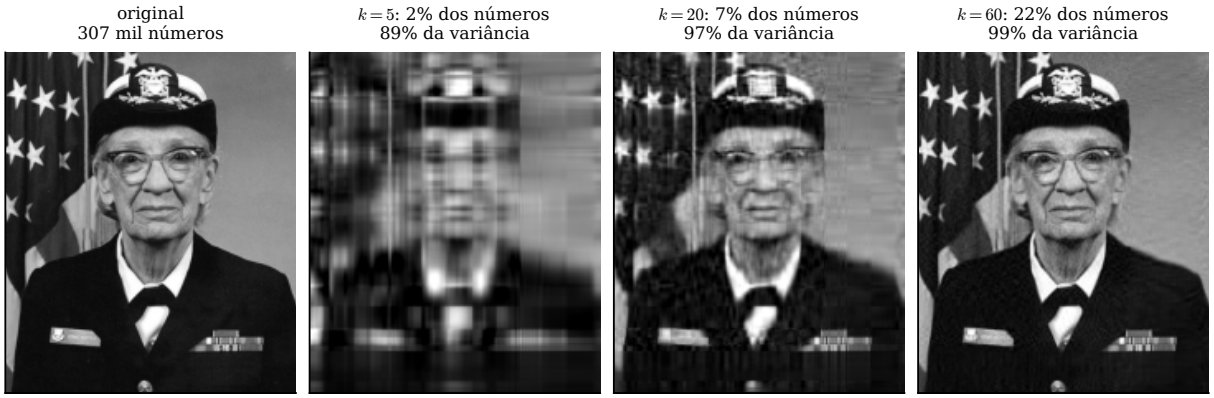


Figura 3: Uma imagem em tons de cinza 600×512 (307 mil números) reconstruída a partir de k componentes. Com $k = 5$ guardamos 2% dos números e já temos 89% da variância — e a imagem, embora borrada, tem a silhueta certa. Com $k = 20$ (7% dos números) o rosto é reconhecível; com $k = 60$ (22%), a diferença para o original quase não se vê. A queda rápida dos autovalores é a razão de tudo isso funcionar.

Atenção

PCA usa variância e produtos internos $\Rightarrow$ é **sensível à escala**. *Padronize* as covariáveis antes (Aula 06), senão variáveis de maior magnitude dominam os componentes artificialmente — trocar metros por milímetros numa coluna muda o resultado. E PCA é *não supervisionado*: a direção de maior variância *não* é necessariamente a mais útil para prever um Y. É perfeitamente possível que o sinal preditivo esteja justamente no componente de menor variância, que você acabou de descartar.

4 Além do linear: Kernel PCA e t-SNE

4.1 Kernel PCA

PCA só captura estrutura *linear*. Como o PCA pode ser escrito em termos de produtos internos, aplica-se o **truque do *kernel*** (Aula 09): substituem-se os produtos internos por um kernel K, fazendo PCA num espaço de *features* de alta dimensão. Resultado: componentes **não lineares**, que capturam subvariedades curvas. É literalmente o mesmo truque da Aula 09, aplicado a outro problema — e essa recorrência não é coincidência: qualquer método que só dependa de produtos internos admite a mesma generalização.

4.2 t-SNE

O **t-SNE** (*t-distributed Stochastic Neighbor Embedding*) é uma técnica *não linear* feita para **visualização** (2D/3D). A ideia:

- converte distâncias em **probabilidades de vizinhança** no espaço original (kernel gaussiano) e no espaço reduzido (distribuição t de Student, de caudas pesadas);
- ajusta as posições no espaço reduzido minimizando a divergência de **Kullback–Leibler** entre as duas distribuições;
- assim, *preserva a estrutura local* (vizinhos próximos continuam próximos), revelando agrupamentos.

O principal hiperparâmetro é a **perplexidade** (ordem de grandeza do número de vizinhos considerados).

Ideia central: por que a distribuição t

O detalhe do nome tem uma razão precisa. Ao espremer dados de $\mathbb{R}^p$ em $\mathbb{R}^2$, não há espaço para manter todo mundo afastado — pontos moderadamente distantes seriam esmagados uns sobre os outros. A distribuição t, com caudas pesadas, atribui probabilidade razoável a distâncias grandes no espaço reduzido, o que permite ao algoritmo *espalhar* os grupos em vez de amontoá-los. É uma solução para um problema de espaço, não de estatística.

Atenção: Cuidados com o t-SNE

- é **só para visualização** — não use os eixos como *features* para um modelo supervisionado (e note que, ao contrário do PCA, ele nem tem um `transform` para aplicar a dados novos);
- **distâncias e tamanhos** entre grupos no gráfico *não* têm significado — só a vizinhança local é confiável. Dois grupos afastados na figura não são necessariamente mais diferentes que dois próximos;
- é **estocástico** e sensível à perplexidade: rode mais de uma vez e compare. Se os agrupamentos mudam de uma rodada para outra, eles não estão nos dados. Para grandes bases, UMAP é uma alternativa mais rápida.

Em resumo: PCA para *compressão e pré-processamento* (linear, interpretável, com `transform` aplicável a dados novos); t-SNE para *olhar* os dados, e só.

5 Prática em Python

```python
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# PCA como etapa de um pipeline (Aula 06): ajustado SO no treino
pipe = make_pipeline(StandardScaler(), PCA(n_components=0.90), Ridge())

# quanta variancia cada componente explica
pca = PCA().fit(StandardScaler().fit_transform(X))
print(pca.explained_variance_ratio_.cumsum())

# t-SNE: apenas para visualizar. O init explicito importa -- com "pca" o
# resultado e reprodutivel; com "random", cada semente da outra figura
Z = TSNE(n_components=2, perplexity=30, init="pca",
         random_state=0).fit_transform(X)
```

O truque do `n_components=0.90` é útil: em vez de um número de componentes, passa-se a *proporção de variância* desejada, e o `scikit-learn` escolhe k sozinho.

Atenção: qual é, afinal, a gravidade desse vazamento

O PCA aprende direções a partir dos dados, então pela regra da Aula 06 ele é etapa do modelo e tem de entrar no *pipeline*. Mas vale ser preciso sobre a *gravidade*, porque a intuição engana aqui.

O que torna grave o vazamento por seleção de variáveis (Aula 06) é que a seleção **olha o** Y: ela escolhe as colunas que, por acaso, se parecem com a resposta naquela amostra, e é assim que fabrica $R^2 = 0{,}40$ a partir de ruído puro. O PCA **não olha o** Y. Sem a resposta, não há como escolher direções que finjam explicar ruído.

E a medição confirma: o notebook desta aula repete o experimento cruel da Aula 06 – y de ruído puro – em quatro configurações de (n, p, k), e em *nenhuma* delas calcular o PCA fora da dobra influ o R^2. Ele o *piorou*, porque componentes calculados sobre o conjunto todo não são os componentes ótimos de nenhuma dobra de treino. Na hierarquia da Aula 06, portanto, o PCA fica na categoria **leve**, ao lado da padronização — e a conduta continua a mesma, porque corrigir custa uma linha.

Os notebooks `Aula prática E2` e `Exemplo - PCA` ilustram os componentes principais e a variância explicada em dados reais.

Resumo

- Reduzir dimensão serve para visualizar, comprimir, pré-processar e explorar *redundância*.
- **PCA**: direções ortogonais de variância máxima; são os autovetores de $\boldsymbol{X}^\top \boldsymbol{X}$ em ordem de autovalor (Prop. 2.1), obtidos de uma única decomposição (Fig. 1).
- *Proporção de variância explicada* (1) guia a escolha de k: no `superconductivity`, 12 dos 81 componentes dão 90% (Fig. 2).
- PCA = melhor projeção linear no sentido de distâncias; a truncagem da SVD é compressão (Fig. 3).
- *Padronize* antes; e lembre que PCA é não supervisionado — a direção de maior variância pode não ser a mais preditiva.
- **Kernel PCA** e **t-SNE** estendem para o não linear; o t-SNE é *só* para visualização, e distâncias entre grupos nele não significam nada.

Leitura recomendada. [AME] Capítulo 10: §10.1 (PCA e a interpretação como escalonamento multidimensional), §10.1.2 (compressão de imagens, que é a nossa Figura 3), §10.2 (Kernel PCA) e §10.4 (*autoencoders*, a generalização não linear via redes neurais). [ISLP] §12.2 — §12.2.1–12.2.2 (o que são os componentes e a interpretação alternativa), §12.2.3 (proporção de variância explicada, a nossa Figura 2) e §12.2.4, que discute honestamente quantos componentes usar e por que não há resposta objetiva.